

RISC-V LINUX 嵌入式编

程技术 · 第一章

开发环境篇

RISC-V 平台认知 · 开发环境安装 · 镜像烧录与启动

— 1.1 / 1.2 / 1.3 —

目标平台：LicheePi 4A /
TH1520

操作系统：
RevyOS

工具链：rui + GNU
Toolchain

开始之前

本章三节是递进关系：1.1 搭建工具链 → 1.2 烧录系统并建立连接 → 1.3 理解自动化的适用边界。请按顺序完成，不要跳节。每节约需 90–120 分钟（含实验）。

你需要准备什么

主机

Linux x86_64
（推荐 Ubuntu 22.04+）。
macOS /
Windows 备选
但课程命令以
Linux 为准。确
保可联网并至少
有 10GB 可用
磁盘空间。

开发板

LicheePi 4A
（TH1520
SoC）一块。配
套：稳定电源适
配器、USB 数
据线（确认支持
数据传输）、
3.3V USB-TTL
串口模块、网线
或 Wi-Fi。

软件

课程指定
RevyOS 镜像文
件 + ruyi 包管理
器 + RISC-V
GNU
Toolchain。本
章全部安装步骤
在 1.1–1.2 中详
述。

本章你会学到什么

1.1 平台认知与环境安装

理解 RISC-V 平台的六个层
次，安装 ruyi 与交叉编译工

1.2 镜像烧录与开发通道

将 RevyOS 烧录到 LicheePi
4A，完成首次启动检查，建

具链，验证工具链可用性。

立串口 / SSH / SCP 三通道。

1.3 provision 适用边界

理解自动化设备准备的适用场景与风险，建立手工 vs 自动的决策框架。

学习建议：每个操作步骤都有对应的验证方法（「你应该看到...」）。如果某步验证不通过，先查阅本节末尾的常见问题，不要跳过。完整终端日志是实验验收材料——从现在开始养成保存日志的习惯。

RISC-V 平台认知与 ruyi 开发环境安装

学习目标

平台分层

区分 ISA、SoC、开发板、OS、工具链六个层次

接口识别

识别 LicheePi 4A 电源、串口、网络 and 启动介质位置

环境清单

确认硬件、线缆、主机软件状态

ruyi 安装

安装 ruyi 包管理器，更新软件源，查询工具链

工具链验证

确认编译器版本、目标三元组，最小编译验证

架构认知

理解 ruyi 管理工具链、GCC 执行编译的分工

平台分层：从 ISA 到应用

RISC-V 是开放指令集架构（ISA），定义了处理器执行指令的接口。芯片和开发板是不同层次的概念。建立分层思维是后续排查「镜像与板卡不匹配」「工具链架构错误」等问题的基础。



rui：包管理与环境管理入口

`rui` 是 RuyiSDK 的包管理入口。它负责发现和获取 RISC-V 开发所需的软件包与工具链；真正编译代码的是 GCC。两者关系类似「应用商店」与「应用程序」。



环境准备清单

项目	要求	检查方式
----	----	------

主机环境	Linux x86_64（推荐），macOS / Windows 备选	<code>uname -a</code>
目标平台	LicheePi 4A 与可用启动介质	核对板卡丝印和包装型号
USB 数据线	确认支持数据传输（非仅充电线）	替换已知数据线测试
USB-TTL 串口	3.3V 电平模块	核对模块规格
网络	可访问 RuyiSDK 软件源	浏览器或连通性测试
基础工具	<code>curl</code> / <code>wget</code> 、 <code>sha256sum</code> 、 <code>file</code>	<code>command -v curl</code>

常见陷阱

- 部分 USB-C 线仅供电、无数据通道 → 无法识别设备时先换线
- 串口电平必须为 3.3V → 5V 可能损坏板卡
- 工具链前缀应选 `linux-gnu`（Linux 应用）→ `unknown-elf` 是裸机工具链，不适用

操作步骤

1

记录主机环境

`uname -a` / `uname -m` — 确认主机架构。主机与目标板架构可以不同，这正是交叉编译的原因。

2

安装 ruyi 并确认命令位置

`command -v ruyi` 确认 shell 实际调用的文件。 `ruyi --version` 记录版本号。

3

更新软件源并查询工具链

`ruyi update` 更新源。 `ruyi list | grep toolchain` 搜索可用工具链。

4

安装并验证编译器

`riscv64-unknown-linux-gnu-gcc -dumpmachine` 应返回 `riscv64-unknown-linux-gnu`。

5

最小编译验证

编译 `int main(void) { return 0; }` → `file` 检查产物为 RISC-V ELF。正确验收方式是确认目标架构——在主机上 `./a.out` 会因架构不同而报错。

课堂练习

1. 解释「LicheePi 4A 是 RISC-V」表述的准确性问题，用 ISA、SoC、开发板三个术语重新表述。
2. 比较 `linux-gnu` 与 `unknown-elf` 两个目标三元组，说明各自适用场景（Linux 应用 vs 裸机程序）。

本节成果：课程平台分层图 · LicheePi 4A 接口标注 · 个人环境清单 · ruyi 安装路径与版本 · 工具链目标三元组与最小编译验证结果。

镜像烧录、首次启动与开发通道建立

学习目标

镜像烧录

理解镜像结构与分区写入流程，安全烧录 RevyOS

系统检查

启动后验证系统版本、架构、网络和时间

串口通道

建立 USB-TTL 本地调试连接作为备用保障

SSH 登录

配置 Ed25519 密钥登录替代密码认证

SCP 传输

双向文件传输 + sha256sum 校验

通道选择

区分串口与 SSH 的适用场景，故障时选对工具

镜像烧录：分区级数据写入

烧录工具将引导程序、内核、设备树和根文件系统按分区结构写入目标介质。建议的烧录流程：校验镜像 → 识别目标设备 → 严格按发布说明执行 → 确认工具有成功报告。

镜像发布包

└─ 引导程序 ← 上电后第一条指令
└─ Linux 内核与设备树 ← 硬件初始化 + 驱动匹配
└─ RevyOS 根文件系统 ← 用户空间全部内容
 ↓ 烧录（按分区写入）
 eMMC / SD / 其他启动介质
 ↓ 上电启动
 串口日志 → 登录界面

三条通道，三个角色

串口 (UART)

本地调试 · 最后保障线

不依赖网络，直接观察启动日志。接线：
GND-GND、
TX→RX、
RX→TX，3.3V 电平。

SSH

远程主工作通道

加密网络终端，适合日常开发。
Ed25519 密钥
登录替代密码。
首次连接须核对主机指纹。

SCP

文件传输通道

复用 SSH 连接传输文件。上传后板端
sha256sum 校验——两端哈希一致即传输无损坏。

首次启动检查清单

检查
项

命令

期望

系统 版本	<code>cat /etc/os-release</code>	RevyOS
架构	<code>uname -m</code>	<code>riscv64</code>
账户	<code>id</code>	记录当前用户
主机 名	<code>hostnamectl</code>	记录主机名
IP 地 址	<code>ip -br addr</code>	有效 IP
路由	<code>ip route</code>	有默认网关
时间	<code>timedatectl</code>	时间正确（错误会导致 TLS 失败）
软件 源	<code>grep '^deb ' /etc/apt/sources.list ...</code>	<code>sudo apt update</code> 成功

SSH 密钥登录配置

```
# 板端启用 SSH 服务
$ sudo systemctl enable --now ssh

# 主机生成 Ed25519 密钥对
$ ssh-keygen -t ed25519 -C "riscv-course"

# 首次连接—核对主机指纹，确认无误后再输入 yes
$ ssh user@board-ip

# 部署公钥
$ ssh-copy-id user@board-ip
```

```
# 免密验证
$ ssh user@board-ip 'hostname; uname -m'
```

指纹变更警告：可能是开发板重装系统，也可能连错了设备。先核对 IP 和串口中的主机指纹，再清理 `known_hosts` 旧记录。

SCP 双向传输校验

```
# 上传
$ printf 'transfer test\n' > test.txt
$ sha256sum test.txt
$ scp test.txt user@board-ip:/tmp/
$ ssh user@board-ip 'sha256sum /tmp/test.txt'    # 哈希应一致

# 下载
$ scp user@board-ip:/etc/os-release ./board-os-release.txt
```

课堂练习

1. 将「开发板无法启动」拆分为镜像、烧录、供电、启动介质和显示通道五类原因，每类写出一个验证方法。
2. 以下故障分别应优先使用串口还是 SSH：板端没有 IP？sshd 配置错误？系统启动失败？需要批量上传文件？

本节成果：镜像来源与校验记录 · 完整烧录日志 · 系统检查表（OS / 内核 / 用户 / 网络 / 时间 / apt）· 串口接线与参数记录 · SSH 密钥登录验证 · SCP 双向传输哈希校验结果。

ruyi device provision 适用边界

学习目标

provision 概念 理解自动化设备准备的目的、输入、输出和潜在风险	设备查询 使用当前 ruyi 版本查询设备支持状态，不凭名称猜测
六维对比 对比手工烧录流程与自动化 provision 的优劣	风险决策 根据六项检查做出自动或手工的路径选择

provision 是什么

Device provision 是把设备准备成可使用状态的自动化过程，可能包括下载镜像、选择设备变体、写入存储介质或生成操作指引。它的价值是减少重复步骤，但自动化不消除风险——误选磁盘、断电或版本不匹配的风险依然存在。「能运行命令」不等于「适合运行命令」。

手工流程 vs 自动化：六维对比

维度	手工流程 (§1.2)	自动化 provision
----	-------------	---------------

控制粒度	每步可控，可随时停止	流程封装，中断后需恢复
适用条件	有镜像和烧录工具即可	设备必须在软件源中明确列出
误操作风险	人为确认每一步，可发现异常	自动化写入可能跳过确认步骤
可调试性	高，每步可单独验证	低，失败后需拆解自动化步骤
重复效率	低，需重复执行相同命令	高，一条命令完成多步
建议场景	首次使用、设备未列出、数据风险高	设备明确支持、重复部署、空白介质

六项风险检查

在运行任何写入操作前逐项确认：

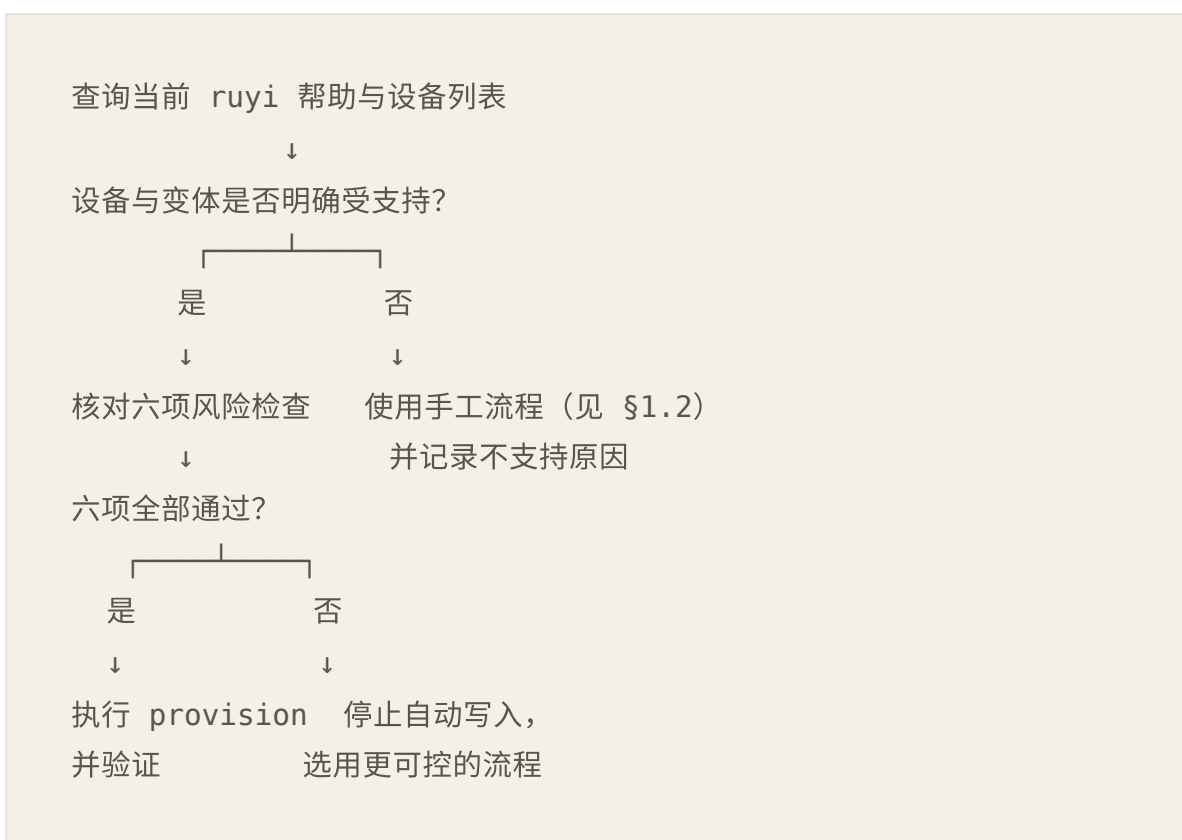
- ① 设备是否在当前软件源中明确列出？
- ② 变体是否与实物一致？
- ③ 写入目标是否唯一且可确认？
- ④ 重要数据是否已备份？

⑤ 供电和数据连接是否稳定？

⑥ 是否已准备串口或其他恢复通道？

关键原则：六项检查全部确认后，方可执行自动 provision。任一项存疑都应停止自动写入，改用 §1.2 手工流程。提交「当前版本不支持该设备」并描述手工替代方案，同样是合格的实验结论。安全优先于自动化。

决策路径



场景决策练习

场景 A

设备完全匹配，
有重要数据

→ 手工流程
(先备份)

场景 B

设备未列出，名
称相似

→ 手工流程，
记录不支持原因

场景 C

设备列出 + 空
白介质

→ 可考虑
provision

本节成果：当前 ruyi 版本 device/provision 帮助记录 · 设备支持查询结果 · 六项风险检查表 · 自动或手工路径的决策记录与理由。